

Introduction

- Cost-aware Dynamic Workflow Scheduling (CADWS) in the cloud involves dynamic workflow DAGs and frequently changing VM resources.
- Existing DRL methods struggle to jointly model complex DAG dependencies and rapidly evolving VM dynamics for CADWS scheduling.
- We propose GATES, a novel and effective DRL method that can capture DAG dependencies and adapt to dynamic VM variations well.

Preliminary of CADWS

Challenges: CADWS features dynamic workflow arrivals, diverse workflow DAG patterns, and varying VM availability.

Workflows: Each workflow is a DAG; tasks follow precedence constraints, and execution time depends on workload and VM speed.

Goal: Minimize total cost, consisting of VM rental fees and SLA penalties.

Methodology

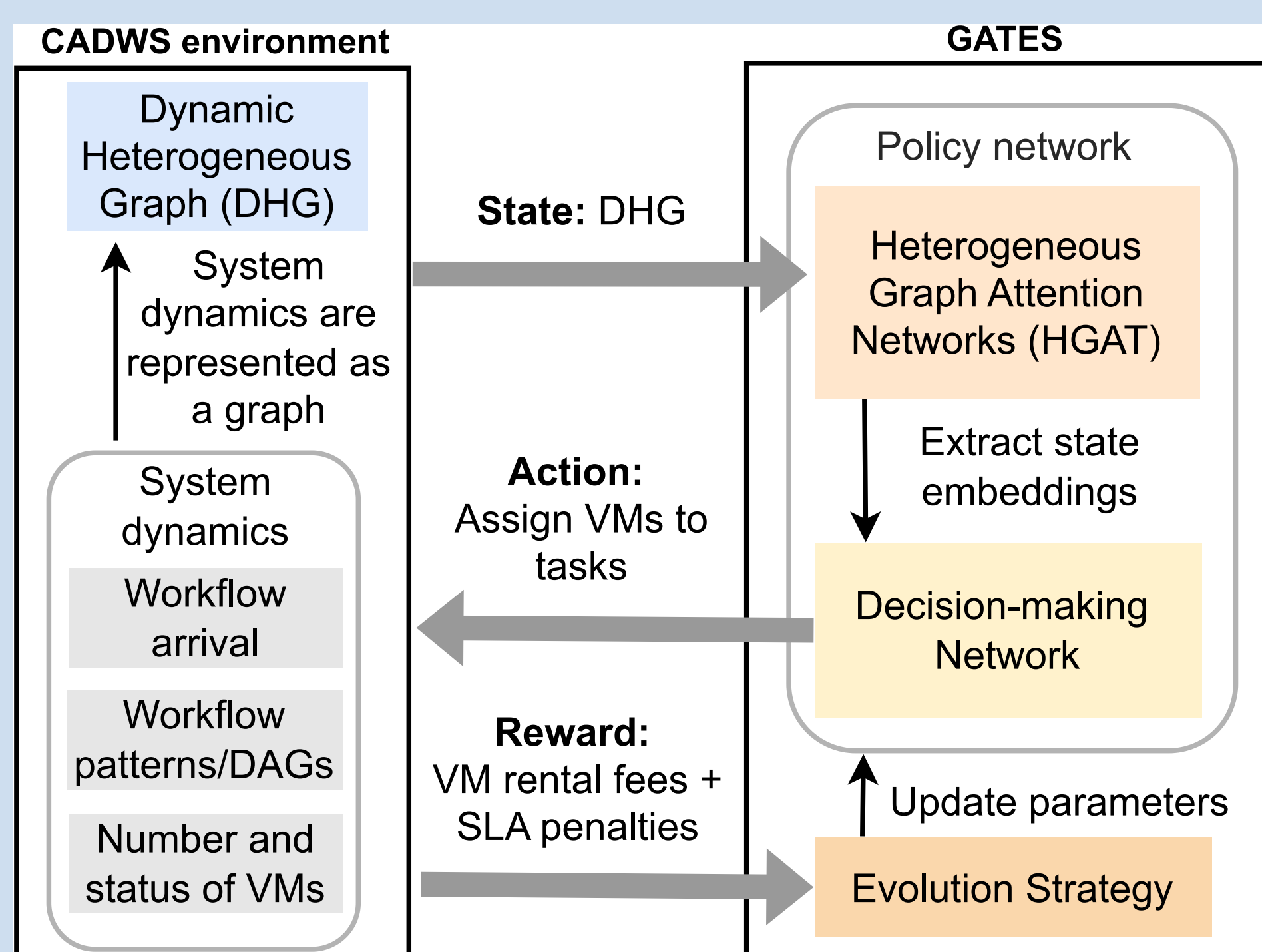
MDP Formulation: Model CADWS as an MDP with iterative scheduling decisions.

State Representation: A novel DHG encodes the current scheduling state to capture the dynamic natures of CADWS.

Graph Processing: An effective HGAT processes the DHG to learn state embeddings.

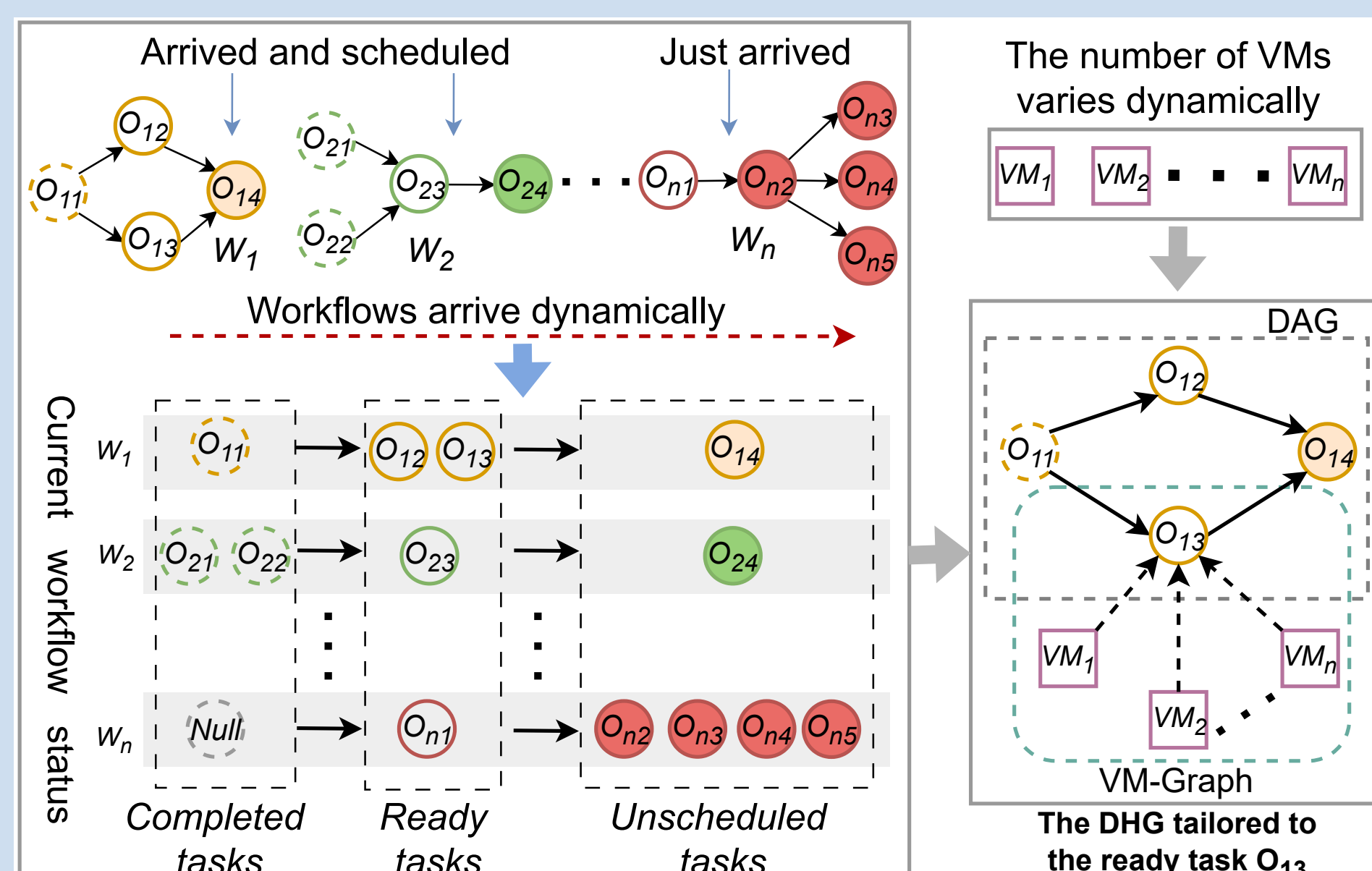
Decision-making: Select the best VM based on learned embeddings.

Policy training: Improve the policy with Evolution Strategy (ES).

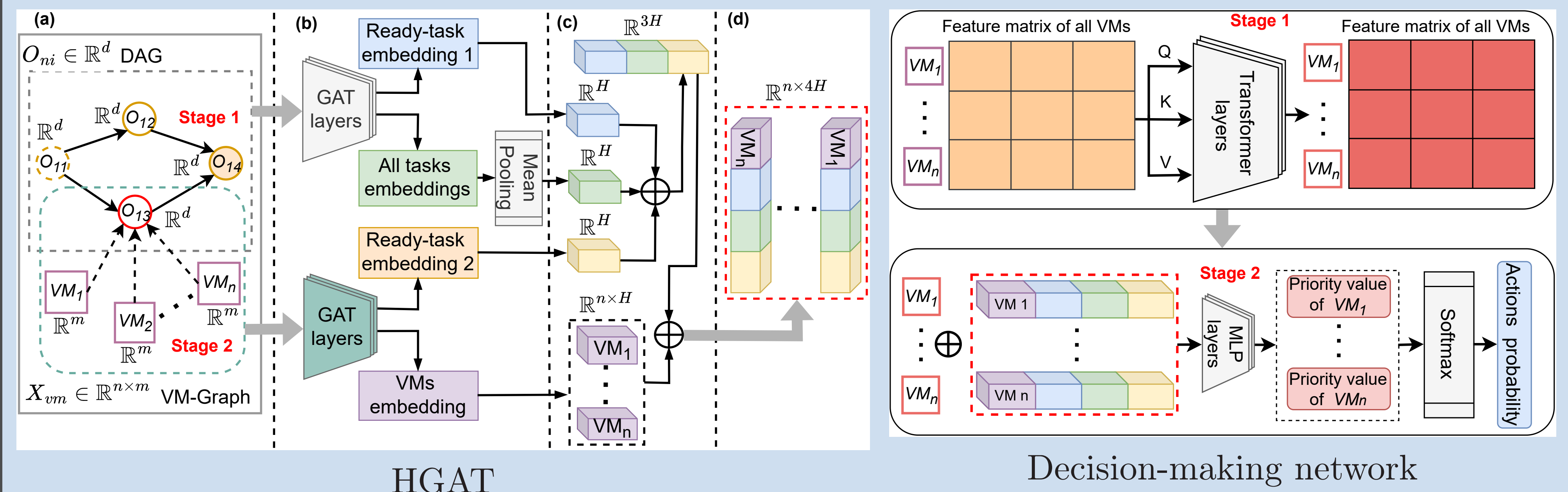


DHG

Dynamically changing VMs are integrated into the workflow DAG to construct an on-demand DHG for each ready task, providing a global view of how all tasks and available VMs influence its scheduling.

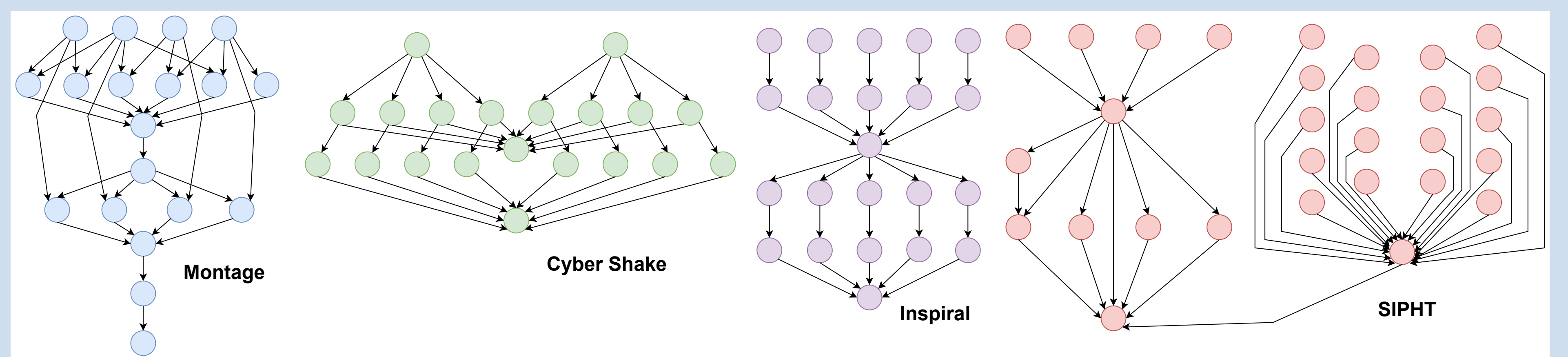


Policy network of GATES



- HGAT encodes workflow DAG topology and VM–task relations within the DHG into state embeddings by using Graph Attention Networks (GAT).
- The Decision-making network maps HGAT-encoded state embeddings to VM selection probabilities for scheduling decisions.

Workflow types

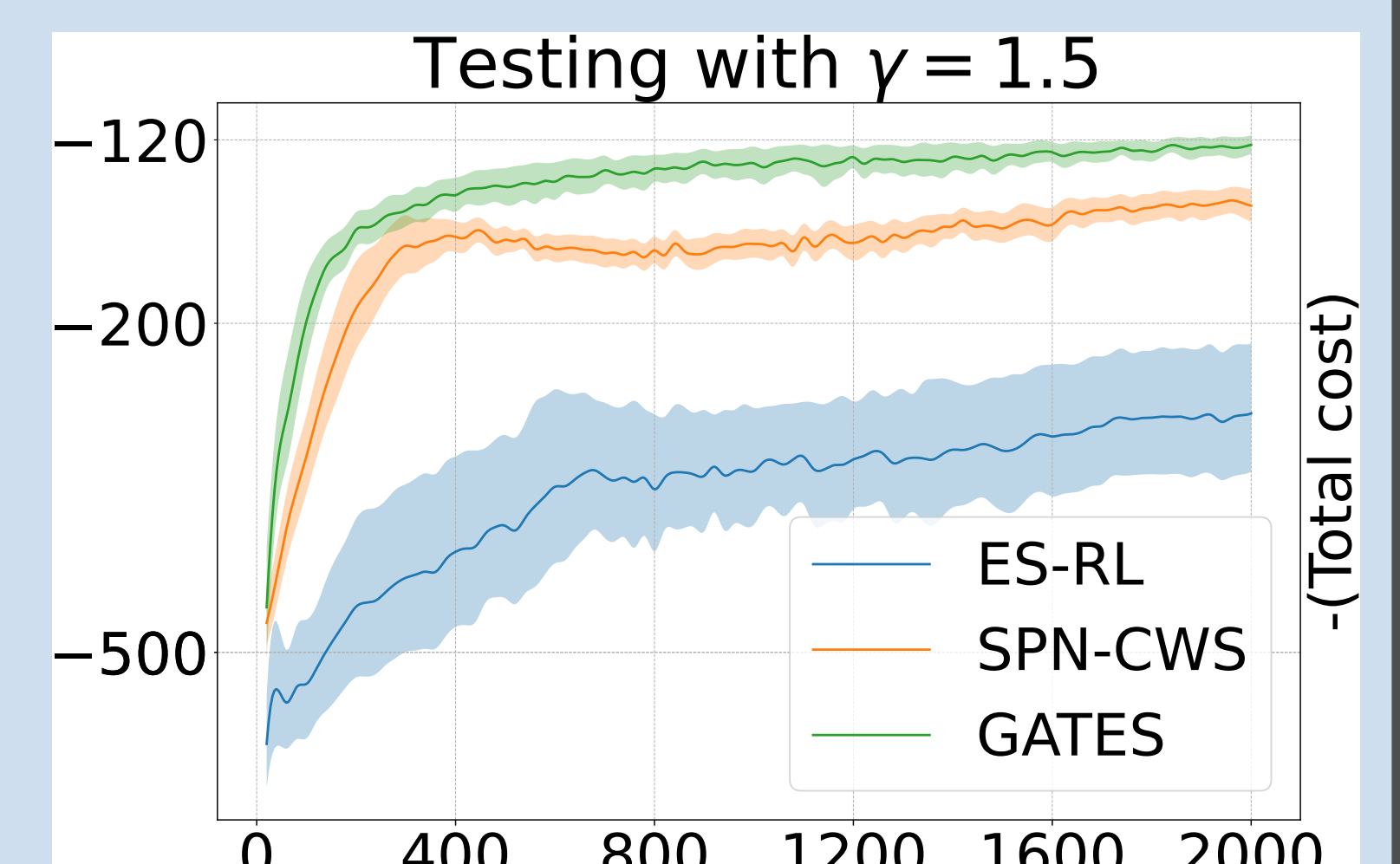
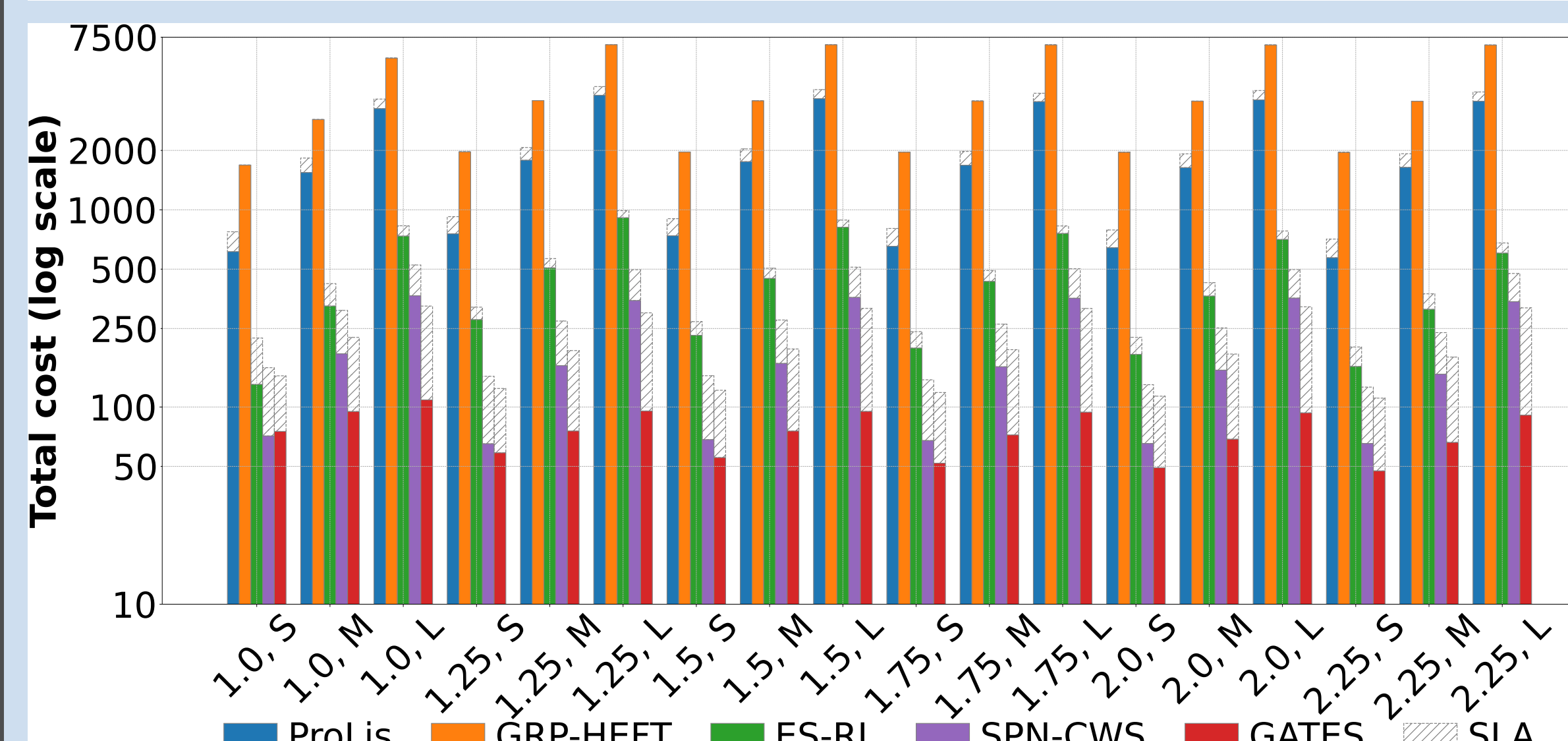


We investigate four types of dynamically arriving workflows, each modeled as a DAG with heterogeneous tasks and precedence constraints.

Results

Scenarios	ProLis	GRP-HEFT	ES-RL	SPN-CWS	GATES (our)
(1.00, M)	773.69	1685.01(-)	223.68(51.51)(+)	158.28(14.45)(+)(+)(+)	143.95(18.94)(+)(+)(+)(+)
(1.00, M)	1829.25	2867.64(+)	422.26(100.92)(+)(+)	308.93(75.88)(+)(+)(+)	225.41(37.21)(+)(+)(+)(+)
(1.00, L)	3641.79	5873.20(+)	829.64(262.91)(+)(+)	524.83(163.15)(+)(+)(+)	324.93(65.65)(+)(+)(+)(+)
(1.25, M)	923.15	1967.31(-)	320.75(83.69)(+)	143.12(15.73)(+)(+)(+)	124.14(7.10)(+)(+)(+)(+)
(1.25, M)	2068.78	3578.04(+)	565.53(213.00)(+)(+)	272.77(60.65)(+)(+)(+)	193.26(24.06)(+)(+)(+)(+)
(1.25, L)	4213.29	6868.22(-)	989.79(458.04)(+)(+)	496.63(155.02)(+)(+)(+)	300.19(76.29)(+)(+)(+)(+)
(1.50, M)	901.40	1963.39(-)	270.93(67.20)(+)	144.13(18.97)(+)(+)(+)	121.56(7.11)(+)(+)(+)(+)
(1.50, M)	2355.52	3567.97(-)	505.44(224.45)(+)(+)	276.09(60.63)(+)(+)(+)	196.90(35.47)(+)(+)(+)(+)
(1.50, L)	4066.84	6863.62(-)	886.87(380.26)(+)(+)	511.69(161.85)(+)(+)(+)	316.23(99.12)(+)(+)(+)(+)
(1.75, M)	804.33	1959.71(-)	241.09(51.43)(+)(+)	137.36(14.10)(+)(+)(+)	118.45(4.85)(+)(+)(+)(+)
(1.75, M)	1977.10	3560.22(-)	491.97(229.48)(+)(+)	263.12(57.61)(+)(+)(+)	195.25(35.83)(+)(+)(+)(+)
(1.75, L)	3898.42	6854.40(+)	828.06(359.74)(+)(+)	502.67(155.64)(+)(+)(+)	318.84(92.48)(+)(+)(+)(+)
(2.00, M)	789.71	1957.25(-)	225.70(50.89)(+)(+)	129.75(12.72)(+)(+)(+)	113.48(5.30)(+)(+)(+)(+)
(2.00, M)	1921.42	3554.07(-)	427.16(213.71)(+)(+)	251.83(52.01)(+)(+)(+)	185.58(30.66)(+)(+)(+)(+)
(2.00, L)	4024.03	6847.49(+)	780.77(408.14)(+)(+)	497.65(158.52)(+)(+)(+)	322.01(110.05)(+)(+)(+)(+)
(2.25, M)	710.18	1954.71(-)	201.54(50.56)(+)(+)	126.18(13.02)(+)(+)(+)	110.90(5.70)(+)(+)(+)(+)
(2.25, M)	1924.27	3551.31(-)	374.96(190.21)(+)(+)	328.35(48.26)(+)(+)(+)	179.17(24.54)(+)(+)(+)(+)
(2.25, L)	3957.08	6842.88(-)	678.94(396.79)(+)(+)	475.46(139.99)(+)(+)(+)	318.19(112.21)(+)(+)(+)(+)

Table 3: Average (standard deviation) total cost of each algorithm over 30 independent runs.



- GATES achieves the lowest total cost across all CADWS scenarios.
- GATES balances VM cost and SLA penalties better than baselines.
- GATES converges faster and more stably with ES optimization.

Conclusion

We propose GATES, a DRL method that models CADWS via a DHG, leverages a novel HGAT network to learn effective scheduling policies, and uses ES for stable, reward-insensitive training.

Acknowledgements

This work was supported by the AI-SCC & Big Data Group at Victoria University of Wellington.

Paper, Code, LinkedIn

